UNIVERSITY OF PLYMOUTH
School of Engineering, Computing & Mathematics

Lecture 1: Introduction to Full Stack

Full-Stack Development

Mark Dixon

School of Engineering, Computing and Mathematics

Introduction

Today's topics

1. Module introduction
2. HTML
3. CSS
4. Client-side JavaScript – brief introduction, more in subsequent sessions
5. Containers & Docker Desktop

Session learning objectives – **by the end of today's lecture you should be able to:**

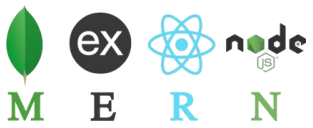
- use **HTML** elements to construct webpage **content**
- use **CSS** rules to achieve the desired **styling** of a webpage
- use **JavaScript** to implement some basic **behaviour**
- use **Docker** to create single container **stack** (Nginx serving HTML, CSS and JS)

Module Overview: Full Stack Developer Skills

- client-side skills (**HTML**, **CSS**, **JavaScript**, **jQuery**, **Angular**, **React**)
- server-side skills (**Node.js** + **Express.js**, ASP.Net, Python, Java, PHP)
- database skills (**MongoDB**, SQL Server, Postgres, Oracle)

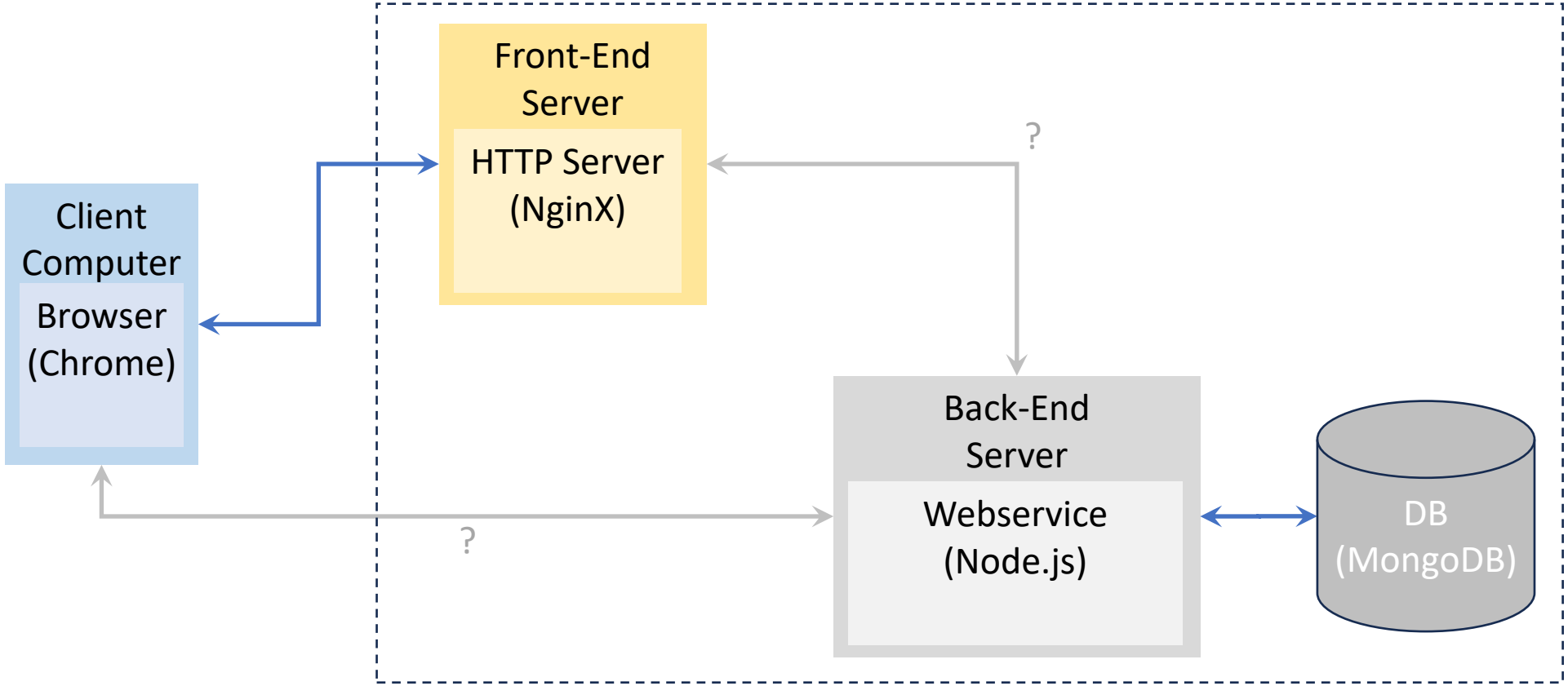
Module Overview: Activity

- **What does MEAN & MERN stand for?**



Full Stack – Context / Component / Container Diagram?

- Sample component architecture (this can vary considerably):



Module Overview: Software

- **Google Chrome** browser (can use others but slides use this)
  https://www.google.com/chrome/index.html
- Microsoft **VS Code** (this is not the full version of Visual Studio)
  https://code.visualstudio.com/
- **Git** (and Git Bash)
  https://git-scm.com/downloads
- **Docker Desktop**
  https://www.docker.com/products/docker-desktop/
- **Node.js**
  https://nodejs.org/en
- **Insomnia** or Postman
  https://insomnia.rest/download https://www.postman.com/downloads/

Module Overview: Aims & Objectives

Aims: To develop an understanding of the role of **advanced scripting techniques** within the development of **dynamic web applications**, and issues in **representing**, **communicating** and **interacting** with **distributed**, **live** and **multi-user** content

Learning outcomes

At the end of the module the learner will be expected to be able to:

1. Apply principles of **object-oriented** and **event-based** scripting as well as **synchronous** and **asynchronous** client-server
2. Demonstrate an understanding of how application content is **represented and communicated** across the web and how this affects the **user experience**
3. **Design**, **implement** and **evaluate/test** dynamic web-based applications

Module Overview: Schedule

- Session 1 Introduction, HTML, CSS, JS, Docker
- Session 2 JavaScript, Node.js
- Session 3 Connecting Containers (JavaScript Objects, JSON & AJAX)
- Session 4 Server-side Persistence (MongoDB & Mongoose)
- Session 5 Node.js Packages & Express
- Session 6 Client-side Frameworks
- Session 7 Client-side & Server-side Testing
- Session 8 Beyond client-server (WebSockets) & DevOps
- Session 9 Client-side Persistence & Concurrency

(subject to change)

Web Application Technology: What is the Web?

The Internet

- publicly accessible worldwide system of interconnected computer networks

The World Wide Web

- A service on the Internet based on four technologies
  - A set of **hypertext pages**
  - Content defined using the HTML mark-up language
  - Client access through servers using HTTP
  - Connected over the Internet using **URIs**

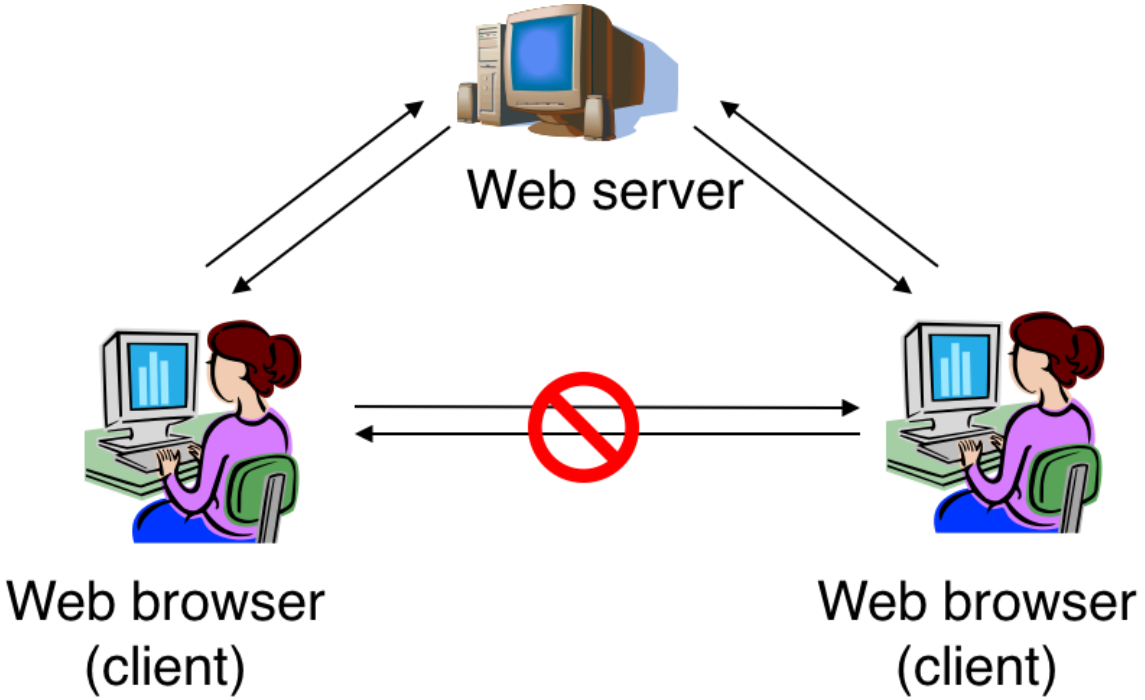
Web Application Technology: Clients & Servers

Client (browser)

- sends **Request** for page / resource
- receives **Response** (data)
- displays /processes **Response** (HTML)

Server

- accepts **Requests** for page / resource
- Finds and processes resources (files)
- sends **Response** (page / resource) back to client



Web Application Technology: separation of responsibility

- Structure – **HTML**
- Style – **CSS**
- Behaviour – **JavaScript**

- Scalable
- Maintainable

HTML: Hyper-Text Markup Language

- Text interspersed with **tags** indicating text properties

```
<element attr1="val" attr2="val">
    Element content goes here…
</element>
```

- e.g., `<b>This will be bold</b>`
- Elements can be **self-closing**, e.g., `<img src="mypic.jpg" />`

HTML: Elements

- element = start tag + content + end tag
  - bold: **This will be in bold**
  - italic: *This will be in italic*

- work like brackets
  - start/open
  - end/close

HTML: Page Structure

- every HTML page has 2 sections:

head
(info)

body
(content)

```
<html>
  <head>
    <title>Test</title>
  </head>
  <body>
    <p>This is a test <b>page</b>.</p>
  </body>
</html>
```

HTML: Attributes

- Some tags need extra information to work:
  - Anchor (hyper-link) element:

```
<a href="nextpage.htm">Next Page</a>
```

attribute (page to jump to)

  - Image element:

```
<img src="Beach.jpg" />
```

attribute (filename of picture to display)

CSS: Style – Cascading Style Sheets

Style

- Layout, colours, font, visibility

CSS

- A language for defining the style of HTML elements
- Rules of a set formal
- Independent of structure, ideally in a separate file

Same structure with different styles

- **Zengarden (http://www.csszengarden.com/)**

CSS: Anatomy

selector { declaration }

- The **selector** is the element within a page to which the style should be applied
- The **declaration block** follows the selector and is delimited by braces { }
- A **declaration** compromises a **property** and a **value** and is terminated with a semicolon – a declaration block can contain multiple declarations

```
body {
    font-family: verdana, sans-serif;
    font-size: 12px;
}
```

CSS: Selectors

- Elements

```
body { }
p { }
h1 { }
```

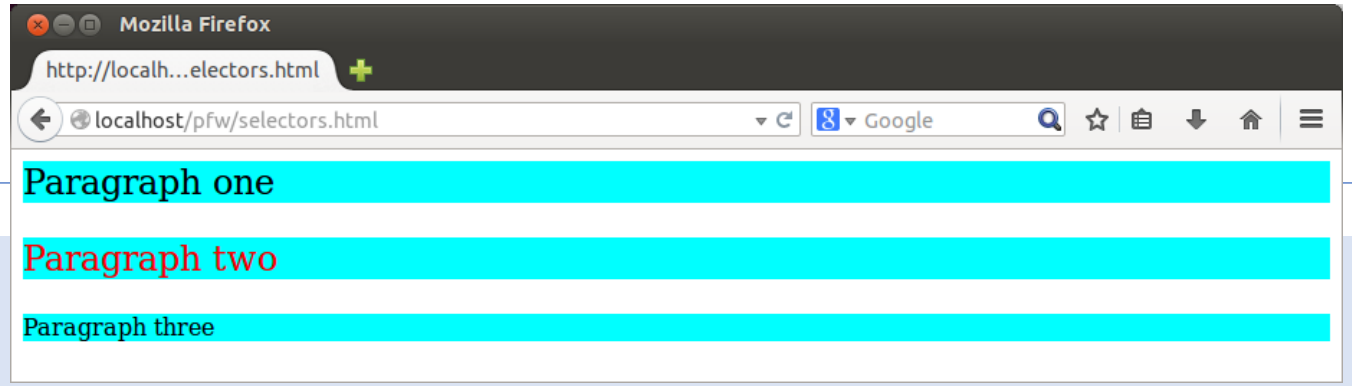
- Classes (can refer to multiple elements) `.class_name`

```
.header { }
.forumpost { }
```

- IDs (can refer to **only one** element) `#element_id`

```
#listitem1
#person47
#errormsg
```

CSS: Selectors




```
<html><head>
    <style>
        p { background-color: cyan; }
        .mypara { font-size: 24px; }
        #parImportant { color: red; }
    </style>
</head>
<body>
    <p class="mypara"> Paragraph One</p>
    <p class="mypara" id="parImportant">Paragraph Two</p>
    <p>Paragraph Three</p>
</body></html>
```

CSS: Selectors

- Attribute

  `selector[attribute]`

  `selector[attribute="value"]`

```
p[id] {}
p[dir="rtl"] {}
```

- Pseudo-class

  `selector:pseudo-class`

```
a:hover {}
a:visited {}
```

- Pseudo-element

  `selector::pseudo-element`

```
p::first-child {}
a::after {}
```

CSS: Rule specificity Activity

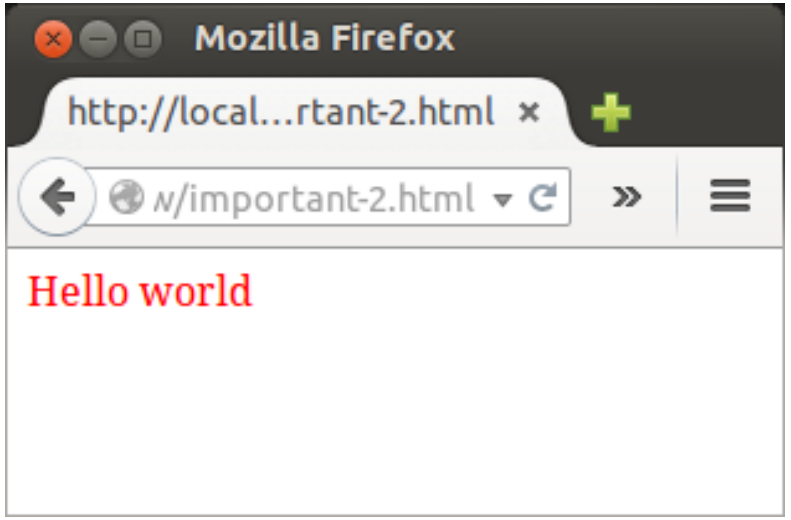
What is Rule specificity?

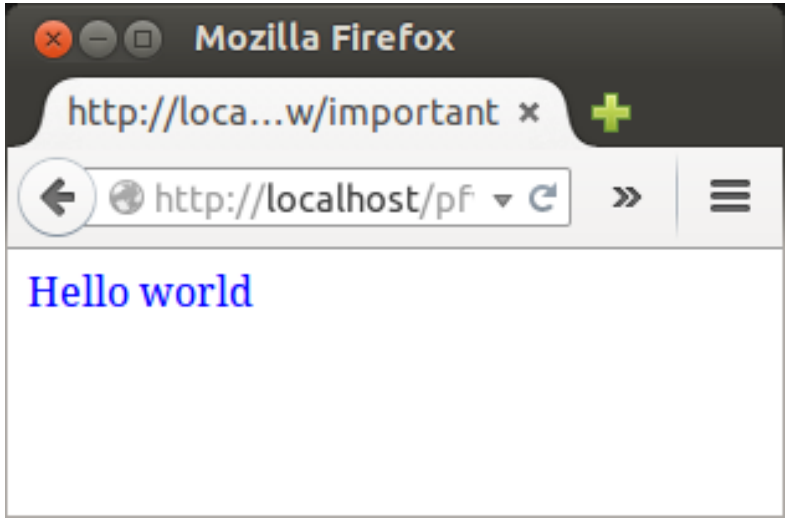
CSS: The Cascade

1. Sort according to explicit weight and origin
   - !important
   - Author, reader, user, agent
2. Specificity
3. Declaration order
   - Later declarations given more weight than early declarations
   - Imported declarations come before those in the importing CSS

```
.mypara { color: blue; }
p { color: red; }
```

```
.mypara { color: blue; }
p { color: red; !important }
```

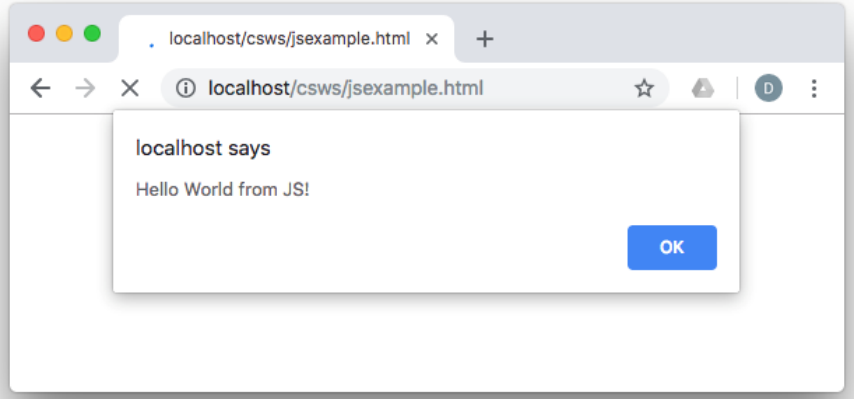




JS: Behaviour

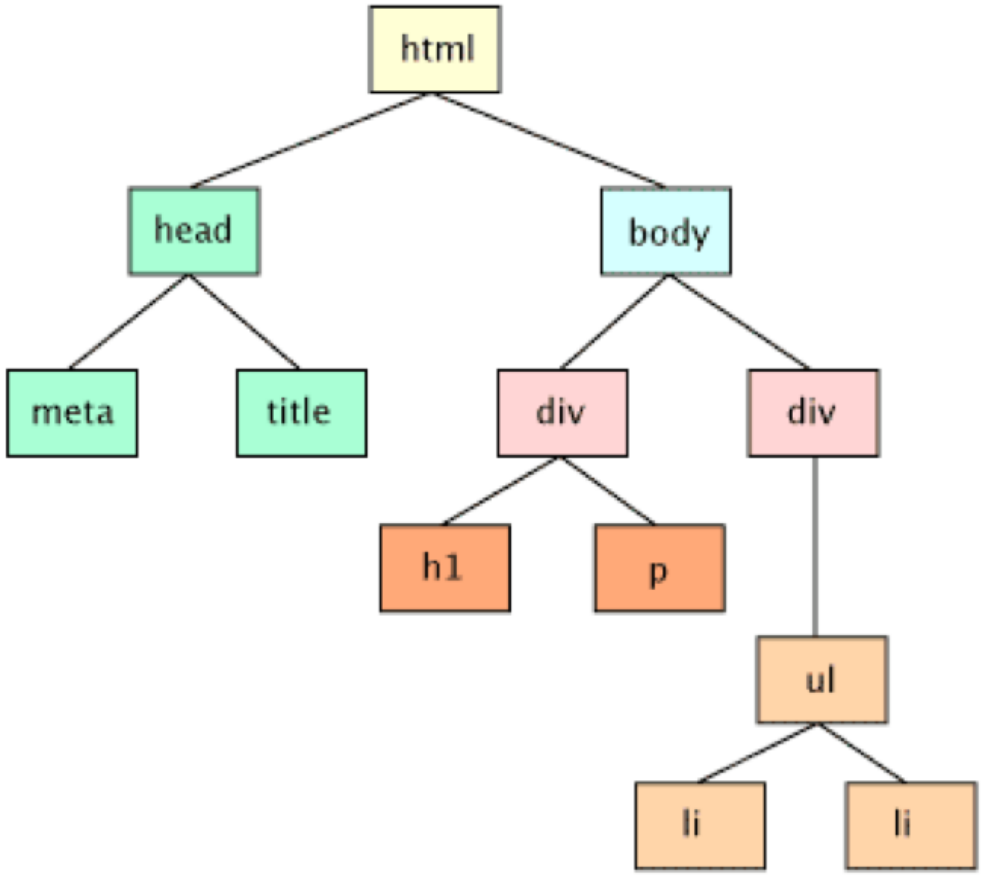
- JavaScript (JS) = programming language
- Until Ajax (IE5 1999) JS was considered a lightweight tool for web developers
- Since then an explosion of frameworks (**node.js**) and libraries (**jQuery**)
- Now dominant as a client-side scripting language

```
<html>
<head>
  <script>
    window.onload = function() {
      alert("Hello World from JS!");
    }
  </script>
</head>
<body></body>
</html>
```



JS: The Document Object Model

- A convention for presenting HTML elements as objects
- Nested HTML elements create a tree structure
- Can be accessed using JS – `document.getElementById()`



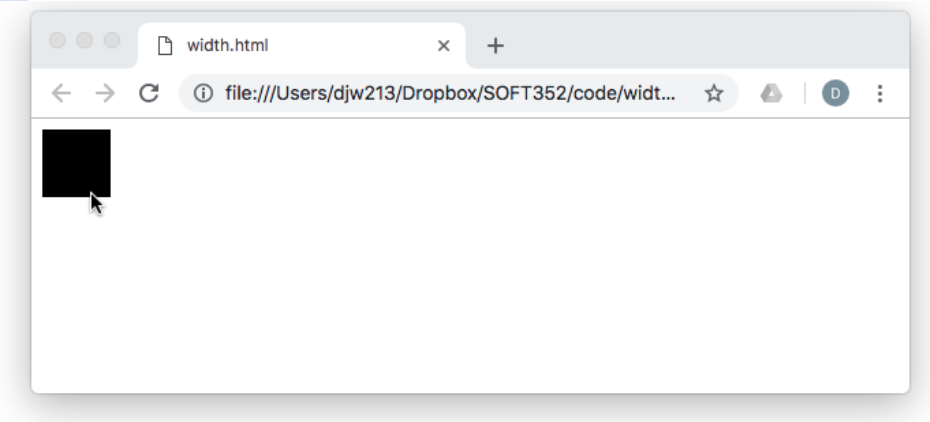
JS: Example

HTML (main.html)

```html
<html>
  <head>
    <link rel="stylesheet" href="main.css">
    <style></style>
    <script src="main.js"></script>
  </head>
  <body>
    <div id="mydiv"></div>
  </body>
</html>
```

CSS (main.css)

```css
#mydiv {
  display: block;
  background-color: #000;
  width: 50px;
  min-height: 50px;
}
```

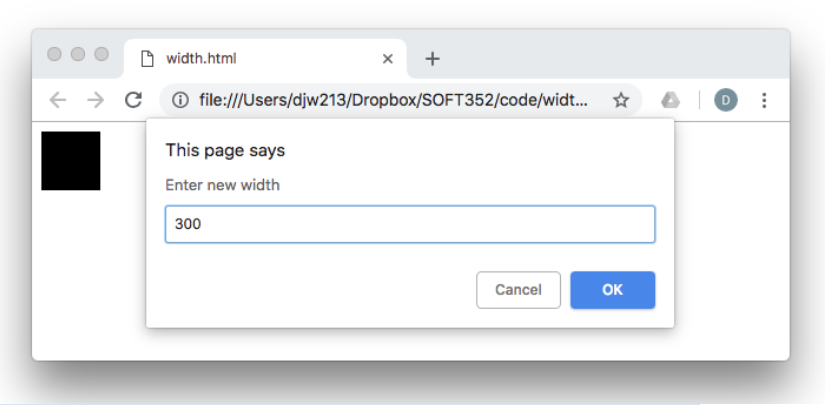


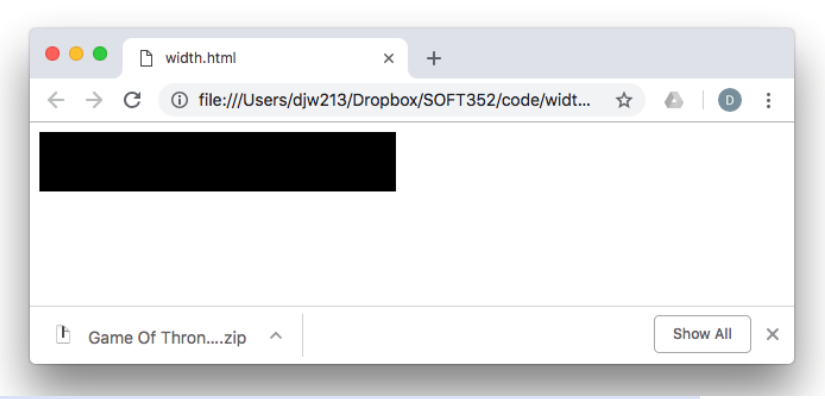
JS: Example

JavaScript (main.js)

```
function changeWidth(newWidth) {
  let elem = document.getElementById("mydiv");
  elem.style.width = newWidth + "px";
}

window.onload = function() {
  let elem = document.getElementById("mydiv");
  elem.addEventListener("click", function() {
    let newWidth = prompt("Enter new width");
    changeWidth(newWidth);
  });
};
```





Containerisation: Docker & Containers

- Problem – dependencies
  'it worked on my computer'
- Solution – installation as code

Virtual Machine

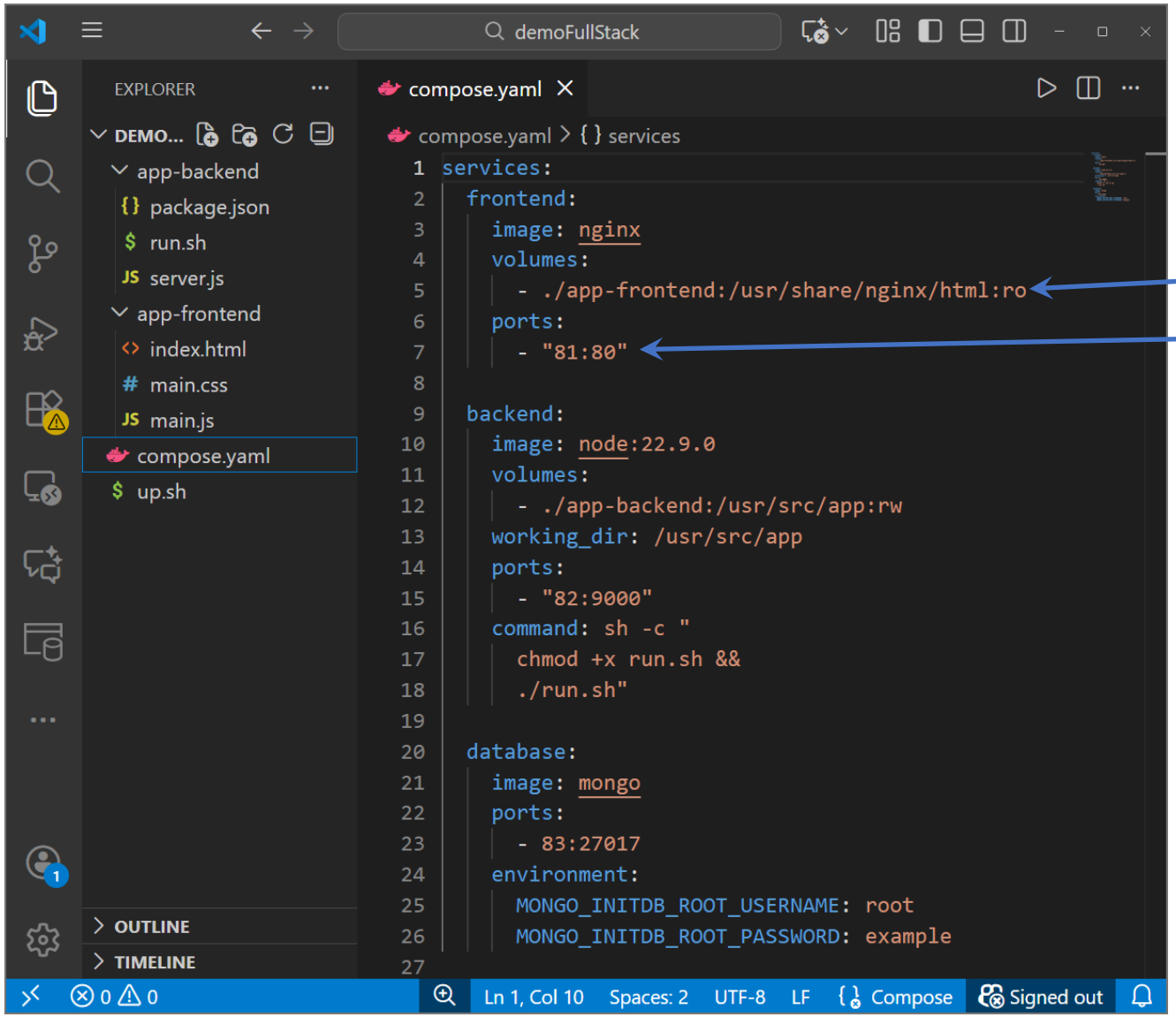
Abstraction of physical hardware
- separate
  - full copy of OS
  - OS kernel
- large (10s of GBs)
- slow boot

Container

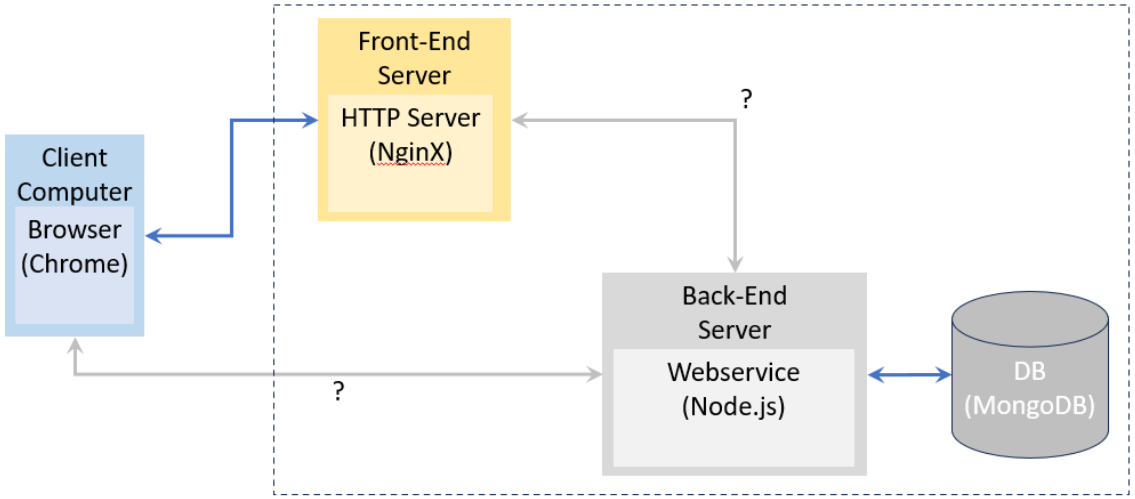
Abstraction of app layer
- separate
  - code
  - dependencies
  - file system
- host OS kernel shared
- small (10s of MBs)
- fast startup

Physical Server

Containerisation: Docker Compose

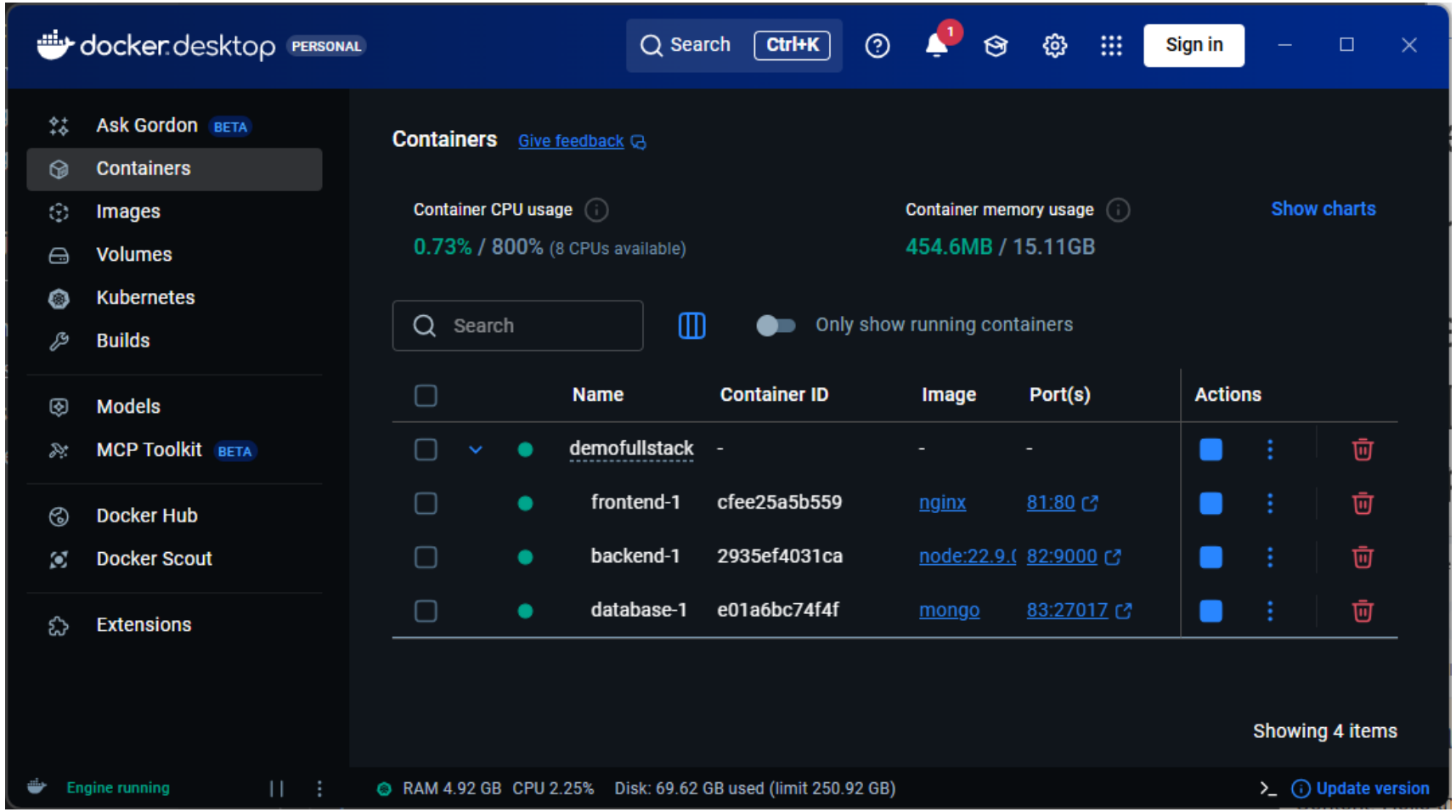


- compose.yaml file defines containers
- each container based on image
- containers can:
  - map [disk] volumes
  - map ports



Containerisation: Docker

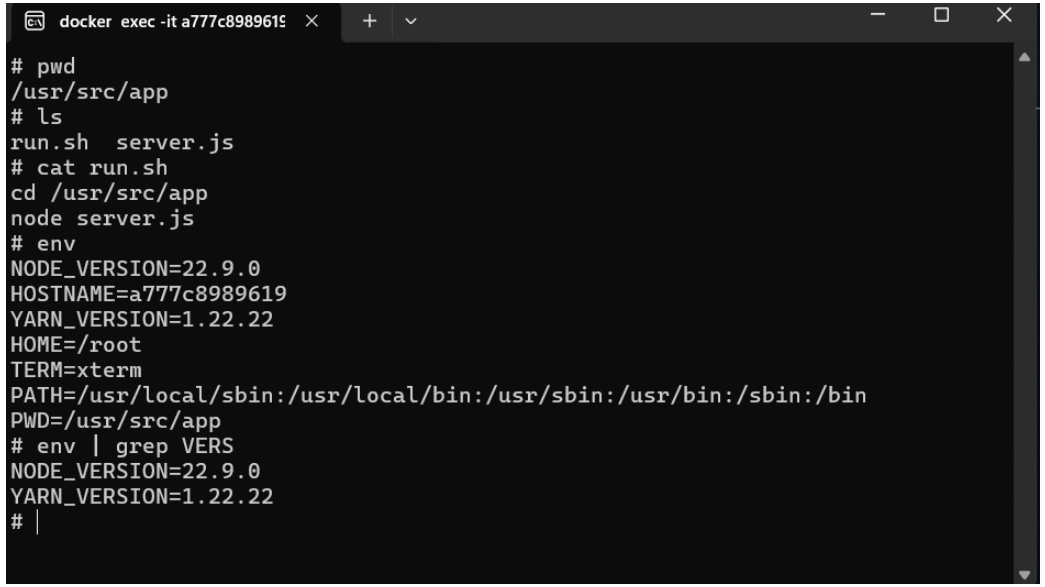
- Containers
- Images
- Port mapping



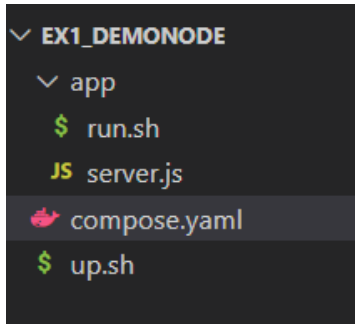
Unix Terminal Commands

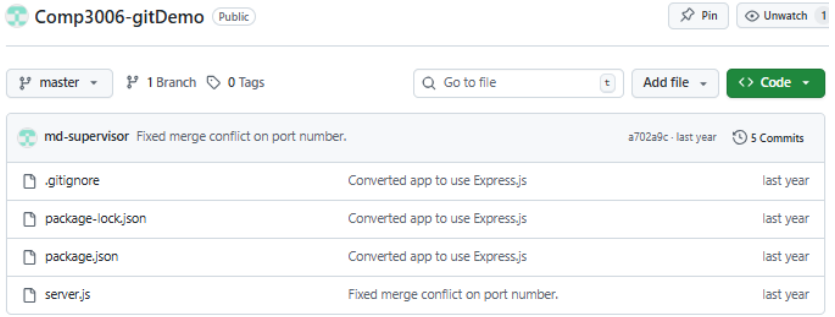
- Docker Containers can be either:
  - **Preferred**: Unix (greater number of images), or
  - Windows
- Useful (for debugging) Unix Commands:

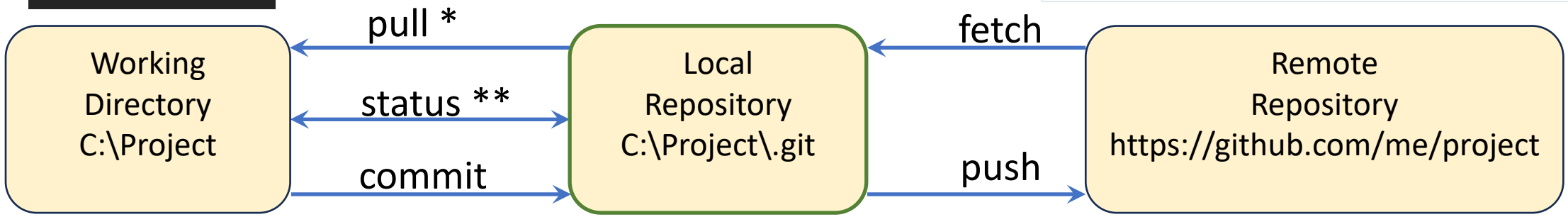
`clear`	clear screen
`pwd`	display current directory
`ls`	list files in current directory
`cd`	change current directory
`cat`	display file contents
`sleep`	wait specified seconds
`env`	display environment variables
`grep`	search files or output of other commands
`curl`	send request over internet
`history`	lists all commands that have been typed (useful for making notes)



Git Commands

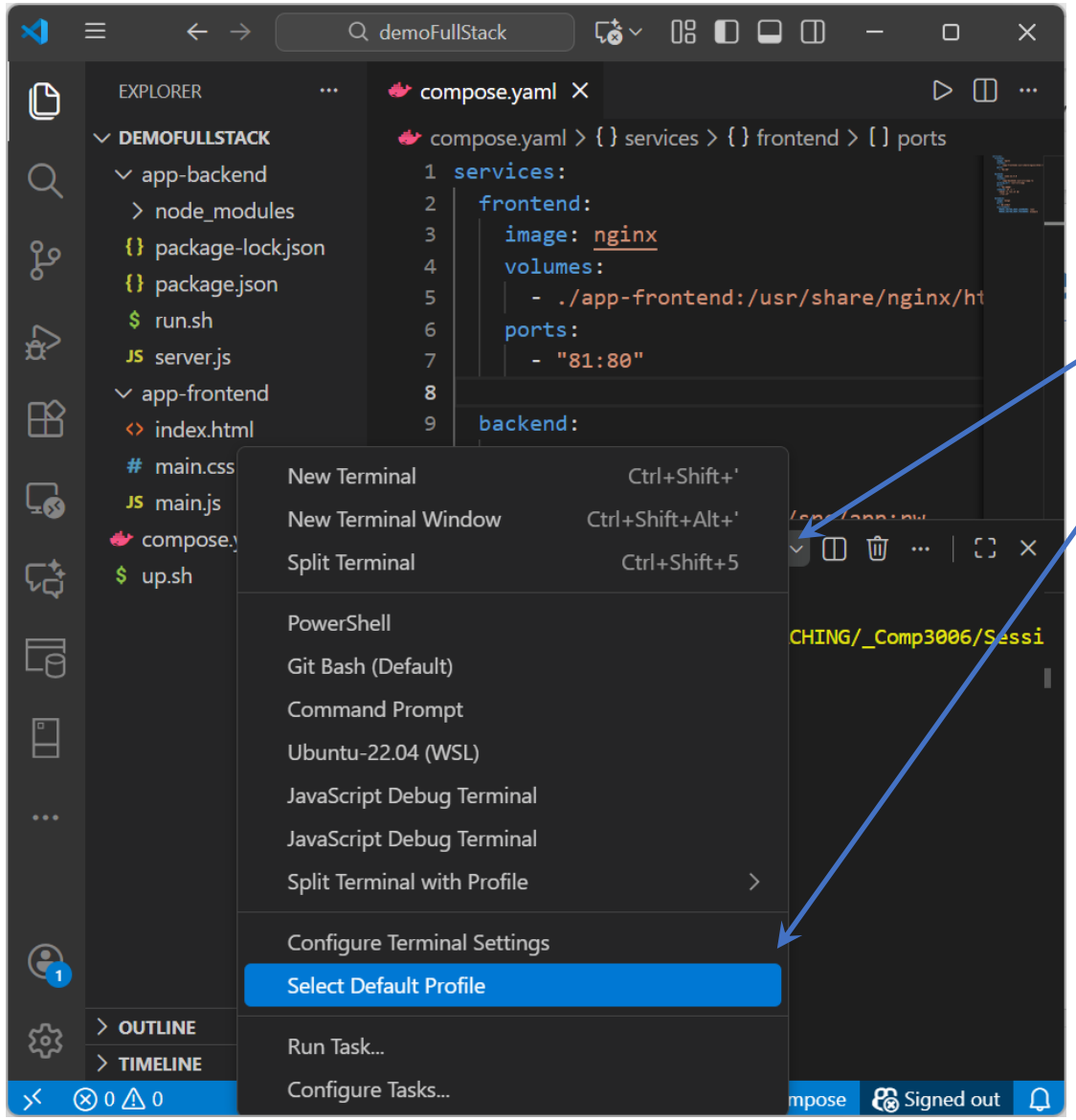






- `git init` – creates the local repository (.git folder, which is hidden)
- `git remote` – facilities to view and change connection to remote repository
- `git status` – compares working directory with local repository (not remote repository) **
- `git fetch` – updates local repo with remote (do this before status)
- `git pull` – updates working directory with changes from local repository (always does git fetch first) *
- `git commit` – copies changes to local repo only (not remote!)
- `git push` – copies changes from local repo to remote repo

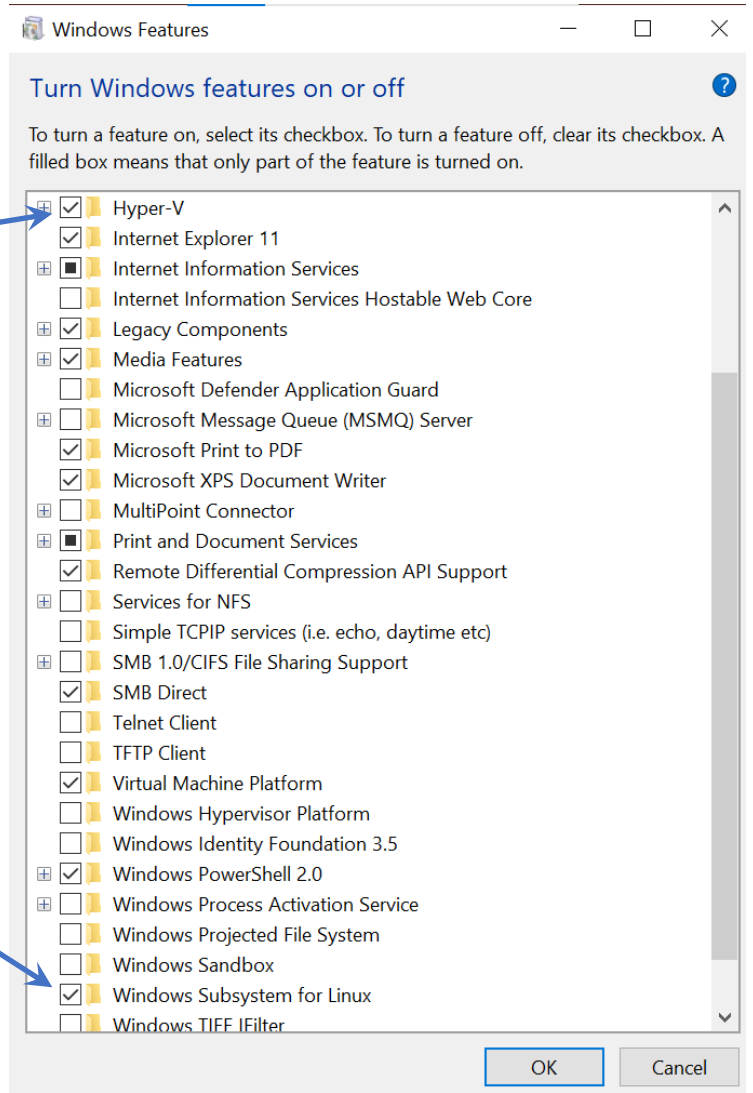
How To: Set VS Code Default Terminal to Git Bash



- Use Unix Terminal (such as Git Bash)
- Set this to be default in VS Code
  - Click down arrow
  - Click Select Default Profile
  - Click Git Bash

How To: Install Docker Desktop (Windows)

- Start Control Panel
- Click '**Programs and Features**'
- Click '**Turn Windows Features on or off**'
- Ensure '**Hyper-V**' is on
  (if this is not shown then continue)
- Ensure '**Windows Subsystem for Linux**' is on
- Download Docker
  https://www.docker.com/products/docker-desktop/
- Install



How To: Install Docker Desktop (Windows)

Then start Git Bash

Set the default WSL version to 2

```
wsl --set-default-version 2
```

You may need to update WSL:

```
wsl --update
```

Install Ubuntu 22.04 LTS distribution:

```
wsl --install Ubuntu-22.04
```

Check installation:

```
wsl -l -v
```

This should return:

```
  NAME                    STATE           VERSION
* Ubuntu-22.04            Running         2
```

https://learn.microsoft.com/en-us/windows/wsl/install

Exercise 1: Single Container stack using Docker

- The following example uses an Nginx image to deploy your HTML, CSS and JS files into a Docker container. You can do everything on the host machine (i.e. your laptop or desktop PC), but this is usually discouraged.
- Start VS Code
- Click File menu > Open Folder item
- Navigate to your desired location (you will need to remember where you have put your project)
- Create new folder '**demoDocker**' (referred to here as the project folder)
- Create new folder '**app**' (inside previous folder)
- Terminal (suggest Git Bash) into project folder (on your host - your laptop or desktop machine)
- Check that Docker is installed (on the host):

```
docker --version
```

- Check that Docker is running (on the host):

```
docker ps
```

Exercise 1: Single Container stack using Docker

- Create new '**index.html**' file (inside app folder)
  Type the following into that file:

```
<html>
    <head>
        <title>Hello</title>
        <link rel="stylesheet" href="main.css" />
        <script src="main.js"></script>
    </head>
    <body>
         Hello there!!!
    </body>
</html>
```

Exercise 1: Single Container stack using Docker

- Create new '**main.css**' file (inside app folder)
  Type the following into that file:

```
body {
    font-size: 120%;
    background-color: aqua;
}
```

- Create new '**main.js**' file (inside app folder)
  Type the following into that file:

```
window.onload = function() {
    alert("Hi!!!");
}
```

Exercise 1: Single Container stack using Docker

- Create new '**compose.yaml**' file (in project folder)
  Type the following into that file (note that the ***indentation*** is critical):

```
services:
  web:
    image: nginx
    volumes:
      - ./app:/usr/share/nginx/html:ro
    ports:
      - "81:80"
```

- Create new '**up.sh**' file (inside project folder)
  Type the following into that file:

```
docker-compose up -d
```

Exercise 1: Single Container stack using Docker

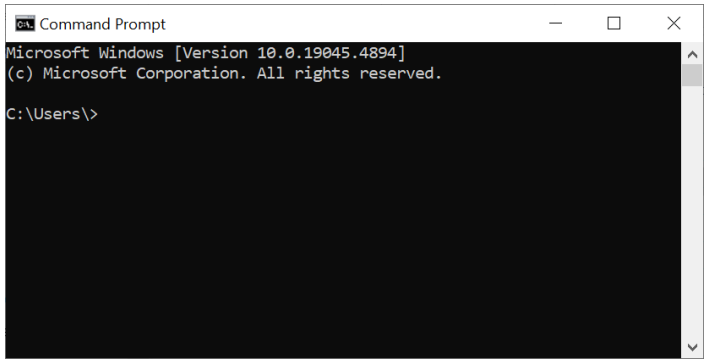
- Terminal into project folder (on your host)
  Start the application up:

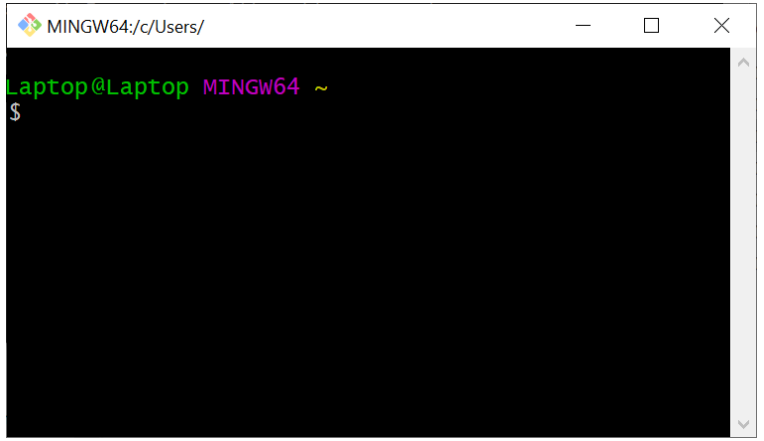
```
source up.sh
```

- On Windows and many Linux systems,
  the word source can be shorted to a dot:

```
. up.sh
```

- Navigate to “http://localhost:81/”, you should see a Hello message.





Note: these are Unix commands, which will not work in Windows Command Prompt or PowerShell (use **GitBash** or a Unix terminal)

Exercise 2: Content & Login

- Modify your previous application to include some more realistic content (such as what you did over the summer or one of your hobbies)
- Add a login page that only allows the user to proceed to the content if the correct username and password are entered
  - at this stage hard code the username and password
  - you would not do this in a real application
    (we can correct this after we cover databases)